



Arquitectura RAG y Métricas de Generación

Diplomatura en IA · UBA/FIUBA · 2 horas

DIPLOMATURA EN IA

POSGRADO

Mapa de la clase

Dos grandes bloques estructuran esta sesión: primero construimos el sistema, luego aprendemos a medirlo.

01

Bloque 0 — Apertura

Pregunta disparadora y promesa de la clase.

02

Sección 1 — Arquitectura RAG

El problema → PoC → Producción → Cierre de sección.

03

Pausa

Descanso entre bloques.

04

Sección 2 — Métricas

Por qué medir → Token Overlap → Faithfulness → Diagnóstico combinado.

05

Bloque Final — Cierre

Síntesis, pregunta de cierre y próxima clase.

BLOQUE 0 — APERTURA

Pregunta disparadora

"Tenemos el PDF del Boletín Oficial, un modelo de lenguaje, y un usuario que hace preguntas. ¿Qué necesitamos en el medio?"

Al terminar van a poder dibujar la arquitectura completa de un RAG en una servilleta, y van a poder decirle a un cliente cuándo el sistema está funcionando bien y cuándo está mintiendo.



SECCIÓN 1 — ARQUITECTURA RAG

El problema que RAG resuelve

· Conceptos base · PoC · Producción · Síntesis



¿Por qué los LLMs solos no alcanzan?

Los LLMs tienen conocimiento estático (corte de entrenamiento) y **no saben nada de los documentos propios de una organización**. RAG es el puente entre el conocimiento del modelo y los documentos específicos del dominio.

Sin RAG

Usuario directo al LLM; conocimiento estático y posible invención



User

LLM

Response



RAG Retrieval
Relevant documents



Grounded
Response



Con RAG

Usuario pasa por RAG; contexto de documentos propios, respuesta fundamentada



Pregunta de anclaje: ¿Cuál es la diferencia entre fine-tuning y RAG? ¿Cuándo usarías cada uno? — *Fine-tuning modifica el modelo (costoso, rígido, para estilos). RAG modifica el contexto en tiempo real (flexible, actualizable, para información factual).*

Las dos etapas de todo sistema RAG

Una PoC no es una arquitectura en producción. Su objetivo es **demostrar que el problema es resoluble** con este enfoque, no que está listo para escalar.

Etapa Offline — Se hace una vez

1. **Ingesta:** Leer PDFs, HTMLs, docs
2. **Limpieza:** Eliminar cabeceras, normalizar Unicode
3. **Segmentación:** Dividir en unidades lógicas (chunks)
4. **Embedding:** Convertir texto en vectores numéricos
5. **Indexación:** Guardar vectores + texto + metadata

Etapa Online — Por cada consulta

1. **Embedding de query:** Misma función que en ④
2. **Retrieval:** Similitud coseno → top-K chunks
3. **Construcción del prompt:** Sistema + Contexto + Pregunta
4. **Generación:** El LLM produce la respuesta fundamentada

Etapas ① y ②: Ingesta y Limpieza

① Ingesta

- Qué entra: PDFs, HTML, Word, CSVs, APIs
- Problema clave: no todos los PDFs son iguales (escaneados vs. nativos)
- Decisión de diseño: ¿extraer página por página o documento entero?
- Herramientas: `pdfplumber`, `BeautifulSoup`

 Ejemplo real: el Boletín Oficial tiene 13 páginas con la cabecera repetida en cada una → hay que eliminarla explícitamente.

② Limpieza

- Qué se limpia: cabeceras repetidas, números de página, caracteres Unicode raros (°, ª, –)
- Herramientas: `regex`, `unicodedata`

Antes:

```
"Edici\u00f3n N\u00b0 21.922\nSalta, lunes  
31...\nDecreto Reglamentario..."
```

Después:

```
"LEY N° 8488. Artículo 1.- Los cursos..."
```

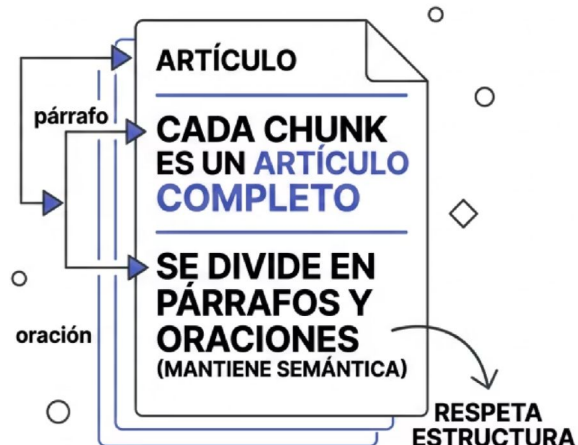

Etapa ③: Segmentación — el momento más crítico

El chunking determina qué unidades de texto va a ver el LLM. Una mala segmentación destruye artículos legales y mezcla documentos distintos en un mismo fragmento.

CHUNK FIJO (150 PALABRAS)



CHUNKING RECURSIVO (JERÁRQUICO)



Impacto directo: Un chunk que cruza dos decretos distintos hace que el LLM mezcle información de ambos, generando respuestas aparentemente coherentes pero factualmente incorrectas.

Etapas ④ y ⑤: Embedding e Indexación

④ Embedding — Texto a vectores

TF-IDF

Frecuencia de términos, sin semántica. Rápido. Ideal para PoC.

Dense (neuronal)

Captura sinónimos y semántica. Lento. Requiere modelo. Para producción.

⑤ Indexación — Qué se guarda

- El **vector** numérico del chunk
- El **texto original** del fragmento
- Los **metadatos**: op_codigo, tipo, ministerio, fecha

Herramienta en PoC: [ChromaDB](#) local en memoria. Los metadatos son la llave del RAG mejorado: permiten filtrar antes del retrieval semántico.

PoC vs. Producción: qué es y qué NO es una PoC

Una prueba de concepto tiene un objetivo específico: demostrar viabilidad del enfoque. No está diseñada para escalar, monitorearse ni actualizarse automáticamente.

PoC

ChromaDB local en memoria

TF-IDF o modelo pequeño

Un script de Python

Sin monitoreo

Sin versionado del índice

Corpus fijo

Producción

Pinecone / Weaviate / pgvector

multilingual-e5-large

Microservicio con API REST

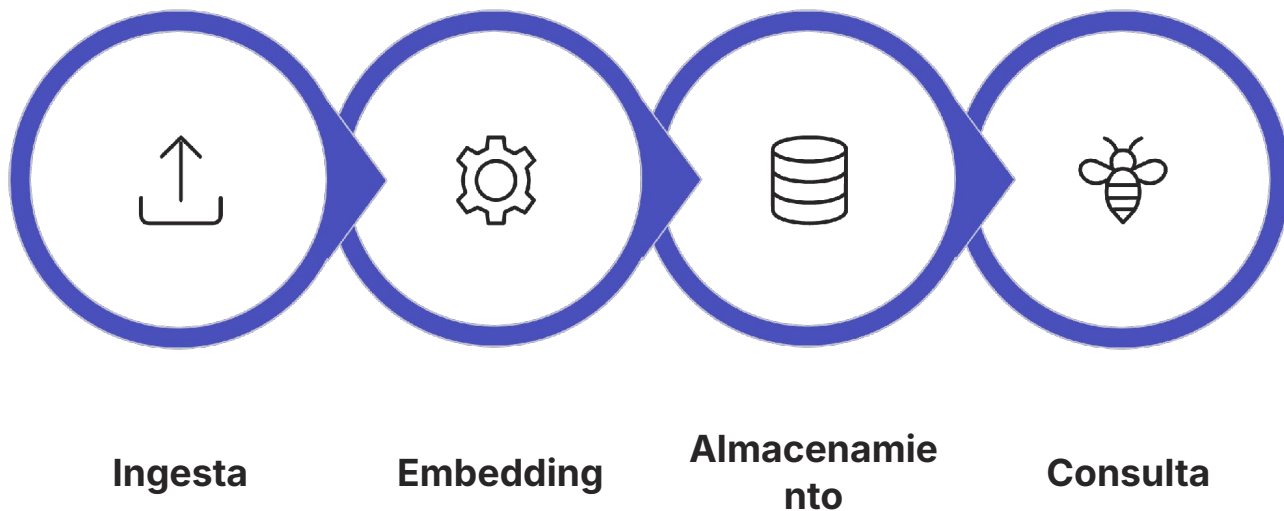
Logging, trazas y alertas

Índice versionado y reproducible

Corpus actualizable con triggers

En producción no alcanza con que funcione

Necesita escalar, actualizarse, monitorearse y recuperarse de fallos. La arquitectura en operación añade capas que la PoC ignora deliberadamente.



Tres diferencias clave: PoC vs. Producción

1. Retrieval híbrido vs. solo vectorial

En producción se combina **BM25** (keywords exactas) con **Dense** (semántica), fusionados con RRF y re-ranking. "Decreto 168" es una keyword exacta que el embedding puede no encontrar si hay variaciones de escritura.

2. Actualización incremental del índice

En PoC se reconstruye el índice completo manualmente. En producción: **upsert incremental** (solo los documentos nuevos), versionado del índice y rollback si la nueva versión empeora las métricas.

3. Observabilidad real

En PoC: `print()` en el notebook. En producción: **logging estructurado** (latencia, tokens, score de retrieval), alertas si Context Faithfulness cae bajo umbral y dashboard de métricas en tiempo real.

El loop de mejora continua

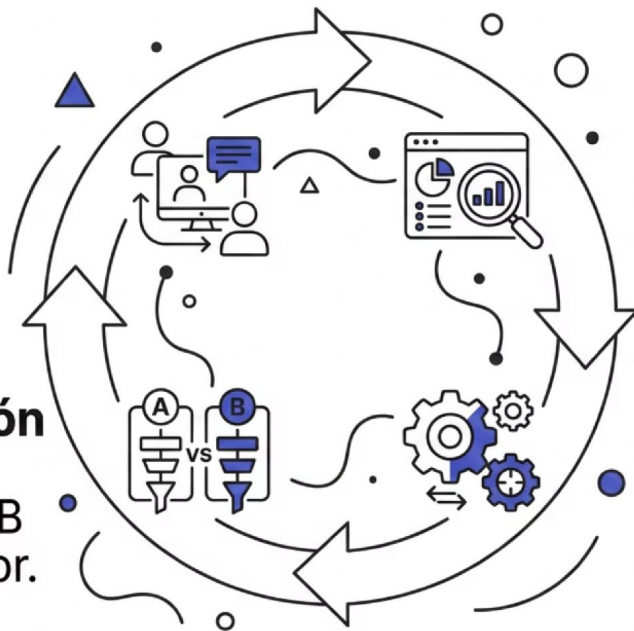
Un RAG en producción nunca está "terminado": las métricas degradadas activan un ciclo de diagnóstico y corrección que produce versiones sucesivamente mejores del sistema.

Usuarios en producción:
generan consultas y feedback.

Nueva Versión RAG:
despliegue A/B
test vs. anterior.

Sistema de evaluación:
calcula Overlap,
Faithfulness,
Latencia, Costo.

Acción Correctiva:
define mejor chunking,
embedding,
prompt.



Una sola pregunta para recordar todo

¿Qué parte del sistema controla si la respuesta es incorrecta?



Problema de Retrieval

El chunk que llegó al LLM **no contiene la respuesta**. Revisar segmentación, embedding y similitud.



Problema de Generación

El chunk **tiene la respuesta** pero el LLM no la usó. Revisar el prompt y la temperatura.



Problema de Ingesta

El texto estaba **mal extraído del PDF**. Revisar limpieza y preprocesado.



Las métricas de la Sección 2 te dicen exactamente cuál de los tres es el problema.

Pausa

20 minutos · Volvemos con Métricas de Generación





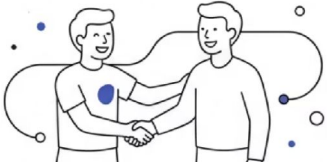
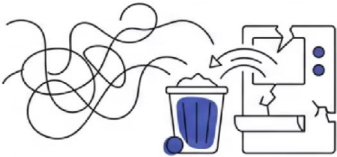
SECCIÓN 2 — MÉTRICAS DE GENERACIÓN

¿Cómo saber si el RAG está funcionando?

45 minutos · Por qué medir · Token Overlap · Context Faithfulness · Diagnóstico combinado

Un RAG puede recuperar bien y aun así mentir

El problema central: un sistema puede recuperar el chunk correcto y generar una respuesta incorrecta, o recuperar el chunk incorrecto y el LLM dar una respuesta que parece correcta pero es inventada. Sin métricas, no hay forma de distinguir los cuatro escenarios.

DIAGNÓSTICO RAG		
	RETRIEVAL CORRECTO	RETRIEVAL INCORRECTO
RESPUESTA CORRECTA	 SISTEMA RAG FUNCIONANDO BIEN	 SUERTE O ALUCINACIÓN COHERENTE
RESPUESTA INCORRECTA	 LLM NO USÓ EL CONTEXTO	 FALLO COMPLETO: BASURA ENTRA BASURA SALE

Token Overlap responde:

"¿La respuesta tiene lo que debería tener?"

Context Faithfulness responde:

"¿La respuesta viene del contexto o el LLM se lo inventó?"

Métrica 1: Token Overlap

Definición: Contamos cuántas palabras importantes de la respuesta esperada aparecen en la respuesta generada. Las "palabras clave" son sustantivos, nombres propios y números — se excluyen stopwords (de, la, el, en, y...).

$$\text{Token Overlap} = \frac{| \text{palabras_clave}(\text{respuesta_generada}) \cap \text{palabras_clave}(\text{respuesta_esperada}) |}{| \text{palabras_clave}(\text{respuesta_esperada}) |}$$

Numerador

Palabras clave de la respuesta generada que **también aparecen** en la respuesta esperada (intersección).

Denominador

Total de palabras clave de la **respuesta esperada** (ground truth).

Rango

Valor entre **0.0** (ninguna palabra coincide) y **1.0** (coincidencia perfecta).

Token Overlap = 1.0 Score perfecto

Pregunta: ¿Quién fue designado titular del Registro Notarial N° 11?

Respuesta esperada (ground truth)

"Bruno José Bocanera, D.N.I. N° 31.904.457"

Palabras clave: {Bruno, José, Bocanera, DNI, 31904457} → 5
tokens

Respuesta generada

"El Escribano Bruno José Bocanera, con D.N.I. 31.904.457, fue designado titular del Registro Notarial N° 11 en la ciudad de Salta."

Intersección: {Bruno, José, Bocanera, DNI, 31904457} → 5 de 5

1.0

Token Overlap

Todas las entidades clave de la referencia están presentes en la respuesta generada.

Token Overlap = 0.17 ⚠ Score parcial

Pregunta: ¿Cuáles son las sanciones de la Ley 8488?

Respuesta esperada

"Apercibimiento y multa de entre 50.000 y 100.000 unidades tributarias"

Palabras clave: {Apercibimiento, multa, 50000, 100000, unidades, tributarias} → **6 tokens**

Respuesta generada (retrieval pobre)

"La Ley 8488 establece sanciones para cursos no autorizados en la provincia de Salta."

Intersección: {sanciones} → 1 de 6

0.17

Token Overlap

El LLM sabe que hay sanciones pero no recuperó los valores específicos. El retrieval entregó el chunk equivocado.

⚠ El LLM habla con confianza de "sanciones" pero omite los montos concretos. Síntoma clásico de retrieval incorrecto: llegó el chunk de contexto general, no el de los artículos sancionatorios.

Token Overlap = 0.0 Alucinación numérica

Pregunta: ¿Cuál es el DNI de Alberto del Milagro Bonifacio?

Respuesta esperada

"D.N.I. N° 25.122.446"

Palabras clave: {25122446} → 1 token

Respuesta generada (alucinación)

"El Suboficial Mayor Alberto del Milagro Bonifacio tiene D.N.I. N° 27.465.890 según el decreto."

Nota: ese DNI pertenece a Figueroa Rueda, no a Bonifacio.

Intersección: {} → 0 de 1

0.0

Token Overlap

El LLM encontró un decreto de retiro similar (mismo template) pero confundió los datos personales. Boilerplate no discriminado en el retrieval.

 Un DNI incorrecto en un sistema legal es un error crítico. Este escenario exige umbrales de Token Overlap más altos que en otros dominios.

Cuándo Token Overlap falla

Token Overlap es una proxy económica y efectiva para dominios con entidades nombradas (números, DNIs, fechas). Pero tiene limitaciones importantes que hay que conocer.

Sesgo por longitud

Puede dar scores altos si la respuesta generada es muy larga y contiene muchas palabras, aunque el contenido clave sea marginal.

No captura cambios de sentido

Si el LLM usa las mismas palabras pero invierte el significado ("no está autorizado" → "está autorizado"), el score puede ser alto y la respuesta, incorrecta.

No maneja sinónimos

"Multa" y "sanción pecuniaria" son equivalentes, pero Token Overlap las trata como distintas.

- ✓ Para este contexto (dominio legal con entidades nombradas: números, DNIs, fechas) funciona muy bien como proxy económico. No requiere un LLM-judge ni llamadas a APIs externas.

Métrica 2: Context Faithfulness

Definición: ¿Cuánto de lo que dijo el LLM estaba realmente en el contexto que le pasamos? Lo que **no estaba en el contexto es alucinación**, aunque sea factualmente correcto.

$$\text{Context Faithfulness} = \frac{| \text{palabras_clave}(\text{respuesta_generada}) \cap \text{palabras_clave}(\text{contexto_recuperado}) |}{| \text{palabras_clave}(\text{respuesta_generada}) |}$$

Token Overlap compara contra...

La **respuesta correcta** esperada (ground truth). Mide si el LLM respondió bien.

Context Faithfulness compara contra...

El **contexto que recibió el LLM**. Mide si el LLM usó ese contexto o se lo inventó.

 Las dos métricas miden cosas distintas y **se necesitan las dos** para tener un diagnóstico completo del sistema.

Context Faithfulness = 1.0 Fiel al contexto

Pregunta: ¿Hasta cuándo dura la afectación de la Sra. Salazar Arce?

"Autorizar la afectación de la Sra. Salazar Arce Carolina Marianela D.N.I. N° 30.221.491 al Concejo Deliberante de la Ciudad de Salta desde el 01 de enero de 2025 hasta el 31 de diciembre de 2025 o mientras duren las necesidades de servicio."

Respuesta generada

"La afectación de Carolina Salazar Arce dura hasta el 31 de diciembre de 2025."

Palabras clave: {afectación, Carolina, Salazar, Arce, dura, 31, diciembre, 2025}

Resultado

Intersección con el contexto: todas presentes.

El LLM **solo usó** lo que estaba en el fragmento recuperado. No añadió información externa ni inventó detalles.

1.0

Context Faithfulness

Respuesta completamente anclada al contexto recuperado.

Context Faithfulness = 0.31 ▲ Alucinación parcial

Pregunta: ¿Quién firmó el Decreto 167?

Contexto recuperado

"SÁENZ - Villada - López Morillo"

Respuesta generada

"El Decreto 167 fue firmado por el Gobernador Sáenz, el Ministro Villada, López Morillo y el Secretario de Gobierno Rodrigo Fernández."

Análisis

Palabras en el contexto: {Sáenz, Villada, López, Morillo} → 4 tokens

Palabras inventadas: {Gobernador, Ministro, Secretario, Gobierno, Rodrigo, Fernández} → el LLM añadió roles institucionales y una persona que no existe en el documento.

0.31

Context
Faithfulness

Rodrigo Fernández directamente no existe en el documento. El LLM completó con su conocimiento institucional previo.

Context Faithfulness = 0.0 El caso más peligroso

Pregunta: ¿Cuántos años de adjunción debe tener un escribano para solicitar registro?

Contexto recuperado (chunk equivocado)

Llegó el decreto de retiro de Figueroa Rueda en lugar del decreto del escribano.

"Disponese el pase a situación de Retiro Voluntario del Suboficial Principal Carlos Cristian Figueroa Rueda..."

Respuesta generada (correcta, pero sin contexto)

"Un escribano debe tener 10 años de adjunción para solicitar el otorgamiento de un Registro Notarial, conforme al Decreto N° 2582/2000."

Intersección con el contexto: {} → ninguna palabra en común.

0.0

Context Faithfulness

El LLM acertó por memoria interna, no por el contexto. Si el decreto se actualiza, el sistema responderá con la versión vieja sin saberlo.

 **Lección clave:** Un RAG que acierta por suerte es tan peligroso como uno que falla. La confianza falsa es peor que el error visible.

Combinando las dos métricas: tabla de diagnóstico

Las dos métricas juntas permiten localizar el problema con precisión quirúrgica. Esta tabla es la herramienta que se llevan a casa.

Context Faithfulness	Alto	Retrieval falla Chunk incorrecto pero el LLM al menos lo usó, revisar chunking. 	Sistema funcionando bien El LLM usó el contexto y respondió correctamente. 
	Bajo	Fallo total Retrieval pobre y el LLM inventó, revisar todo el pipeline. 	LLM acertó por memoria No por contexto, peligroso si el documento se actualiza. 
		Bajo	Alto
		Token Overlap	

Métrica	Tolerable	Objetivo PoC	Objetivo Producción
Token Overlap	> 0.30	> 0.60	> 0.75
Context Faithfulness	> 0.50	> 0.70	> 0.85

Síntesis de la clase

Sección 1 — El mapa de lo que hay que construir

PoC: Ingesta → Limpieza → Chunking → Embedding → ChromaDB
→ Retrieval → LLM

Producción: Todo lo anterior + retrieval híbrido + re-ranking +
monitoreo + loop de mejora continua

Sección 2 — Cómo saber si está funcionando

- **Token Overlap:** ¿Responde bien? (compara contra ground truth)
- **Context Faithfulness:** ¿O está inventando? (compara contra el contexto)
- **Ambas altas:** Sistema confiable
- **Ambas bajas:** Volver al EDA y a la ingesta

Pregunta de cierre: Token Overlap = 0.9 y Context Faithfulness = 0.2. ¿Es un sistema confiable para producción?

❌ **Respuesta esperada:** No. El LLM responde bien hoy porque "recuerda" la información, pero si el documento se actualiza, el sistema seguirá respondiendo con la versión vieja sin saberlo. Faithfulness bajo en producción legal es un riesgo inaceptable.